
DANDI — Discrepancy-Aware Navigation for Distributed Inference over Enterprise Context Graphs

BRIEF ID

brief-0001-dandi-the-walker

STATUS

DRAFT

CREATED

2026-02-16

SECTION 01

Research Brief

01 PROBLEM FRAMING

Enterprise context graphs are the hot infrastructure bet of 2026. Foundation Capital calls them a trillion-dollar opportunity. Glean, Graphlit, Palantir, and a wave of startups are racing to build them — stitching together decision traces, entity relationships, and cross-system state into navigable substrates for AI agents.

Everyone is focused on building context graphs. Nobody has formalized what happens when an agent actually tries to use one.

Here's the problem: enterprise state is inherently fragmented and contradictory. A sales rep promises "free migration" in Slack. The signed contract says nothing about it. The billing system shows a charge for migration services. The CRM lists the deal as closed-won with no special terms. No single source contains the truth. The truth requires walking across all four, noticing the conflict, and inferring what actually happened.

Existing approaches fail at this:

One-shot RAG retrieves chunks that are semantically similar to the query but misses the narrative thread linking a promise to a contract to a billing event. It cannot detect that these three sources disagree.

Long-context ingestion dumps everything into the window. It's cost-prohibitive at enterprise scale, suffers from lost-in-the-middle degradation, and is passive — it doesn't actively hunt for conflicts.

GraphRAG traverses a pre-built knowledge graph but treats it as read-only. It can follow edges but cannot write back reconciled state when it finds contradictions.

The missing layer has two parts. First, there is no standard navigation interface. The “graph” isn’t a graph database — it’s Slack, Jira, HubSpot, Google Drive, billing APIs, contract PDFs. These are heterogeneous systems with different interfaces, auth models, and data shapes. Nobody has defined a minimal set of primitives that lets an agent traverse them as a unified substrate. Second, there is no contradiction-aware reasoning with write-back. Agents read but don’t refactor. When an agent discovers that three sources disagree, there’s no mechanism to write an inferred-state node back into the substrate that says “these sources conflict, here’s the reconciliation, here’s the evidence chain.” Every future query has to rediscover the same conflict from scratch.

Core Bet / Hypothesis

We propose DANDI: a method for stateful navigation and distributed inference over enterprise context graphs. The system has two components.

The Context Shell — a standardized navigation interface that exposes heterogeneous enterprise systems through a small set of shell-like primitives:

- `list_sources` — enumerate available systems of record
- `query(source, params)` — structured query against a specific source
- `open(source, id)` — retrieve a specific record
- `search(query)` — cross-source semantic search
- `trace_link(source, id, direction)` — follow a reference from one source into another
- `write_inference(evidence[], conclusion, confidence)` — write a reconciled state node back into the substrate

The agent operates under a bounded navigation budget (max steps, max tokens), forcing it to traverse strategically rather than exhaustively.

Semantic State Refactoring (SSR) — when the agent detects discrepancies across 3+ sources, it uses `write_inference` to create an inferred-state node that reconciles the conflict with an explicit evidence chain. These nodes are persistent — downstream agents can query them directly instead of re-deriving the same conclusion. This turns enterprise context from a read-only retrieval target into a compiling, self-improving substrate.

The analogy: developers navigate codebases with `ls`, `cd`, `grep`, `open`, and `write` refactored code when they find inconsistencies. DANDI gives agents the same structured traversal for enterprise state.

02 WHAT WE'RE BUILDING

Context Shell specification — the formal primitive set, navigation budget model, and interface contracts for wrapping real enterprise systems (Slack, CRM, document store) as shell-navigable sources.

SSR protocol — the mechanism for `write_inference`, including evidence chain format, confidence scoring, and how inferred nodes are stored and queried by future agents.

DANDI agent — a reference implementation that uses the Context Shell primitives with an LLM backbone to walk real enterprise systems, detect cross-source contradictions, and produce reconciled state.

SynCorp-2026 benchmark — a reproducible evaluation environment using real free-tier services (Slack workspace, HubSpot/Zoho CRM, Google Drive) populated with synthetic company data containing injected multi-source contradictions. This is NOT a document-level benchmark (that's ContraGen). This tests navigation across real heterogeneous APIs.

Discrepancy test harness — an evaluation protocol that scores agents on hidden-conflict discovery rate, evidence chain completeness, token efficiency, and navigation path optimality.

Seed examples to get the team thinking (not exhaustive): A customer was promised 99.9% uptime in a Slack thread. The signed SLA says 99.5%. The monitoring dashboard shows 99.7%. What was the actual commitment, and is it being met? A billing charge for "Enterprise Plan" appeared in January, but the CRM shows a downgrade to "Pro Plan" in December. The support ticket from the customer says they were told the downgrade would take effect immediately. What's the correct billing state? A contract renewal was signed with a 15% discount, but the discount code in the billing system shows 10%. The sales rep's Slack message to the deal desk requested 15%. Where did the discrepancy originate?

03 BOUNDARIES AND NON-GOALS

In scope is the Context Shell primitive specification and reference implementation, the SSR protocol and `write_inference` mechanism, the DANDI agent using an existing LLM (no fine-tuning), SynCorp-2026 benchmark using real free-tier services, and the discrepancy test harness and evaluation protocol.

Out of scope is building the ontology/graph construction layer (that's Agent Builder, a separate team), raw SaaS connectors, OAuth plumbing, or production-grade auth, user-facing dashboards or UI, fine-tuning base models, and scaling beyond the benchmark scenario.

04 SUCCESS SIGNAL

✓ PRIMARY

The DANDI agent autonomously identifies a hidden truth that is contradicted or obscured across at least three distinct real systems (e.g., Slack, CRM, and a contract in Google Drive), produces a correct reconciliation with a traceable evidence chain, and does so with fewer tokens than a long-context baseline that ingests all sources at once.

✓ BENCHMARK ADOPTION

SynCorp-2026 is adopted by at least one external research group within 6 months of release. The Context Shell specification is cited or forked.

✓ COST DISCIPLINE

Total compute and API spend under \$1,000 for the full project lifecycle.

05 KEY ASSUMPTIONS

▪ CONTRADICTIONS ARE SYSTEMIC

Real enterprise state is inherently fragmented and contradictory. The contradiction scenarios in SynCorp-2026 are representative of what actually happens when promises in chat conflict with contracts and billing records.

▪ NAVIGATION BEATS INGESTION

Strategic traversal of enterprise systems via a bounded set of primitives produces more reliable contradiction detection than dumping all sources into a long context window, while consuming fewer tokens.

▪ WRITE-BACK COMPOUNDS VALUE

Inferred-state nodes produced by `write_inference` provide genuine value to downstream agents. A reconciliation done once and stored is more efficient than re-deriving the same conclusion from scratch on every future query.

- **REAL APIS ARE NECESSARY**

Evaluating on real heterogeneous APIs (Slack, CRM, Drive) captures difficulties that document-level benchmarks miss — different data shapes, rate limits, auth models, and reference formats across systems.

- **FREE TIERS ARE SUFFICIENT**

Free-tier access to Slack, HubSpot/Zoho, and Google Drive provides enough API surface for a credible benchmark. Rate limits can be worked around with caching and batching.

- **LLM BACKBONE IS NOT THE CONTRIBUTION**

DANDI works with any capable LLM. The contribution is the navigation interface, the SSR protocol, and the benchmark — not the model.

SECTION 02

Abstract Draft

01 ABSTRACT

Enterprise context graphs — the emerging infrastructure for cross-system decision intelligence — inherit the contradictions of the fragmented systems they unify. A delivery promise in Slack may conflict with a signed SLA, or a billing charge may be inconsistent with a recorded plan change in the CRM. Existing approaches treat these graphs as read-only retrieval targets, leaving cross-source contradictions effectively invisible. We introduce DANDI (Discrepancy-Aware Navigation for Distributed Inference), a method for stateful agent navigation and semantic state refactoring over heterogeneous enterprise systems. DANDI exposes real systems — messaging platforms, CRMs, document stores — through a Context Shell: a minimal set of navigation primitives with a bounded traversal budget. During navigation, the agent can invoke `write_inference` to produce reconciled state nodes when it detects discrepancies spanning three or more sources, compiling high-density, auditable context for downstream agents. We formalize the discrepancy test — resolving hidden truths that require evidence from multiple platforms — and release SynCorp-2026, a reproducible benchmark built on real free-tier enterprise services with injected cross-source conflicts. We show that DANDI improves multi-hop contradiction resolution while reducing token cost versus long-context ingestion, without requiring a pre-built knowledge graph or privileged access to proprietary data. The contribution is twofold: an architecture for navigating enterprise context as a live substrate rather than a flat index, and an evaluation protocol that stresses traceability, efficiency, and correctness across real system boundaries.

SECTION 03

Reading Map

01 CONTENTS

The reading map covers five areas: the enterprise context graph thesis that DANDI positions against, the closest prior work on contradiction detection and agent memory, evaluation infrastructure and benchmark design, the tooling and APIs you'll build with, and risks to watch for. Start with the Hands-On Exercise before reading papers — the exercise builds intuition for the API data shape problem that no paper describes.

02 FOUNDATIONS

Jaya Gupta & Ashu Garg, “AI’s Trillion-Dollar Opportunity: Context Graphs” Foundation Capital, December 2025 <https://foundationcapital.com/context-graphs-ais-trillion-dollar-opportunity/> Read the full post. This is the thesis DANDI builds on. Pay attention to “decision traces” and “systems of record for decisions.” The argument is that context graphs are the next infrastructure layer for AI — DANDI’s contribution is formalizing what happens when an agent actually navigates one and finds contradictions.

“The Context Graph — AI’s Trillion-Dollar Opportunity” RunDataRun, January 2026 <https://rundatarun.io/p/the-context-graph-ais-trillion-dollar> Best analysis of the prescriptive vs. emergent ontology split. Understand both camps — DANDI sidesteps the ontology question entirely by navigating systems as-is rather than requiring a pre-built graph.

Glean, “Context is the next data platform” January 2026 <https://www.glean.com/blog/context-data-platform> Read the section on how they define “process understanding.” This is the industry incumbent view. DANDI’s argument is that Glean and similar tools treat context as a retrieval target, not a navigable substrate with write-back.

Jamin Ball, “Long Live Systems of Record” Clouded Judgement (referenced in Foundation Capital post) The counter-argument that agents don’t kill systems of record, they raise the bar. Important for framing — DANDI agrees with this: the shell navigates existing systems of record, it doesn’t replace them.

“Graph-based Agent Memory: Taxonomy, Techniques, and Applications” arXiv, February 2026 <https://arxiv.org/html/2602.05665> Survey paper covering the full landscape of graph-based agent memory. Read the sections on knowledge memory construction, latent link prediction, and inductive knowledge enrichment. The KG-Agent framework (ref [41]) is the closest to DANDI’s hybrid approach. Useful for positioning in Related Work.

03 CLOSEST METHODS

ContraGen: “A Multi-Agent Generation Framework for Enterprise Contradictions Detection” arXiv, October 2025 <https://arxiv.org/html/2510.03418v1> This is the closest prior work. Read the methodology section carefully. They generate contradictory documents and detect intra/cross-document conflicts. Key gap: they work on documents, not on heterogeneous systems. Their future work section explicitly calls out multi-hop reasoning across heterogeneous sources as unsolved — that’s exactly what DANDI does. Reviewers will ask about ContraGen, so know exactly what they did and what they didn’t.

“Contradiction Detection in RAG Systems: Evaluating LLMs as Context Validators” Amazon, arXiv, March 2025 <https://arxiv.org/html/2504.00180v1> Read the data generation framework and the taxonomy of contradiction types. Important for designing SynCorp’s contradiction injection methodology. Their taxonomy (direct contradiction, temporal inconsistency, partial overlap) informs the five contradiction categories in the benchmark.

ALICE: “Automated Logic for Identifying Contradictions in Engineering” Automated Software Engineering, June 2024 <https://link.springer.com/article/10.1007/s10515-024-00452-x> Formal logic + LLM approach to contradiction detection in requirements. Different domain but the taxonomy of contradiction types is useful. The insight that formal logic can complement LLM-based detection may be relevant for future extensions.

Graphiti/Zep: “Build Real-Time Knowledge Graphs for AI Agents” GitHub <https://github.com/getzep/graphiti> Read their temporal knowledge graph model and the write-back mechanism. Understand how they handle entity resolution and fact expiry. They do memory write-back but not contradiction-aware refactoring — they accumulate facts over time but don’t detect or reconcile conflicts across sources. DANDI’s SSR is the missing piece.

INRAExplorer: “Agentic RAG with Knowledge Graphs for Complex Multi-Hop Reasoning” arXiv, July 2025 <https://arxiv.org/abs/2507.16507> Multi-tool agent architecture over a knowledge graph. Read the tool design and evaluation. Closest to DANDI’s “agent with primitives walking a substrate” pattern, but they work on a pre-built KG of publications, not heterogeneous enterprise systems. DANDI navigates real APIs.

AGENTiGraph: “A Multi-Agent Knowledge Graph Framework” arXiv, August 2025 <https://arxiv.org/html/2508.02999v1> Users can curate and manipulate KGs via natural language. Read the agent pipeline (intent → graph update → knowledge integration). Relevant for understanding how write-back to graphs has been approached — but AGENTiGraph writes to a graph database, not back across heterogeneous enterprise systems.

Microsoft GraphRAG Documentation and paper Read the community summarization approach specifically. Understand how they pre-compute graph structure at ingestion time vs. DANDI’s approach of navigating at query time. GraphRAG is read-only — no write-back, no contradiction awareness.

04 EVALUATION

SynCorp-2026 (to be built) The benchmark DANDI introduces. A reproducible evaluation environment using real free-tier services (Slack workspace, HubSpot/Zoho CRM, Google Drive) populated with synthetic company data containing injected multi-source contradictions. Five contradiction categories: promise vs. contract, billing vs. plan, timeline conflicts, authority conflicts, and status conflicts. Minimum 15 test cases, target 30.

Evaluation metrics: discovery rate (did the agent find the contradiction?), evidence completeness (did it cite all relevant sources?), reconciliation accuracy (is the inferred conclusion correct?), token efficiency (vs. long-context baseline), navigation efficiency (steps used vs. budget), and path quality (human evaluation).

05 SYSTEMS

Slack API documentation <https://api.slack.com/> Focus on conversations.history, search.messages, and the data model for threads. This is the “unstructured” source — messages, threads, reactions. The Context Shell wrapper needs to handle Slack’s thread model and pagination.

HubSpot API (free tier) or Zoho CRM API <https://developers.hubspot.com/> Focus on contacts, deals, tickets, and the search/filter APIs. Understand rate limits on free tier. This is the “structured” source — fields, records, relationships. The CRM choice (HubSpot vs. Zoho) depends on which free tier is less restrictive.

Google Drive API <https://developers.google.com/drive> Focus on file search, metadata retrieval, and content download for Docs/PDFs. This is the “document” source — contracts, SLAs, proposals. The wrapper needs to handle both metadata queries and content extraction.

LangGraph documentation <https://www.langchain.com/langgraph> For building the agent’s state machine. Understand the state graph model, conditional edges, and tool integration. LangGraph is the framework for the DANDI agent loop — it handles state, branching, and tool calls.

06 RISKS

Free-tier API rate limits. Slack, HubSpot, and Google Drive all have rate limits on free tiers that may be more restrictive than expected. If rate limits are too tight, the benchmark runs will be slow or require caching. Mitigation: test limits in Phase 0, have a backup CRM option ready, implement response caching.

Synthetic scenario credibility. SynCorp-2026 uses injected contradictions in a fictional company. Reviewers may question whether these scenarios are realistic enough. Mitigation: use realistic enterprise language, get feedback from someone with actual enterprise support experience, and ground contradiction types in documented real-world patterns from ContraGen and the Amazon contradiction taxonomy.

ContraGen overlap. The ContraGen team could publish a multi-source extension before DANDI. DANDI’s differentiator is real APIs (not documents) and write-back (not just detection) — maintain this gap clearly in the paper.

Write-back safety. Hallucinated inference nodes can poison future queries. Mitigation: confidence thresholds, multi-evidence requirements (3+ sources minimum), and garbage collection or retraction protocols for low-confidence nodes.

Scope creep into graph construction. The temptation to build the ontology or graph construction layer is strong. DANDI’s boundary is clear: the shell navigates systems as-is. Graph construction is the Builder team’s problem. Enforce this boundary.

07 HANDS-ON EXERCISE (DO THIS BEFORE PHASE 1)

Before building DANDI, each team member should complete this exercise:

1. Set up a free Slack workspace, a free HubSpot account, and a Google Drive folder.
2. Manually create a 3-source contradiction scenario (e.g., a promise in Slack that conflicts with a CRM record and a contract in Drive).
3. Write a simple script that calls all three APIs and retrieves the relevant records.
4. Try to answer the question “What was actually agreed?” using only the API responses.

This exercise builds intuition for the navigation problem and the data shape challenge before you touch any LLM code. Document every API pain point, rate limit surprise, and data shape mismatch — these become the requirements for the Context Shell.

08 SUGGESTED READING ORDER

1. **HANDS-ON EXERCISE FIRST**
 - before any papers.
2. **FOUNDATION CAPITAL POST**
 - understand the industry thesis.
3. **CONTRAGEN**
 - understand the closest prior work and the gap DANDI fills.
4. **GRAPH-BASED AGENT MEMORY SURVEY**
 - understand the landscape.
5. **GRAPHITI/ZEP**
 - understand existing write-back approaches.
6. **API DOCUMENTATION**
 - (Slack, CRM, Drive) — set up the three services.
7. Everything else as needed during Phases 1-2.

Suggested pace: 2 weeks on reading, then start Phase 0 of the experimental blueprint.

SECTION 04

Experimental Blueprint

Team size: 3. Timeline: approximately 5 months (18 weeks). Compute budget: under \$1,000. LLM backbone: Claude Sonnet for development, frontier model for final benchmarks. Infrastructure: real free-tier enterprise services (Slack, CRM, Google Drive) — no simulated APIs.

01 PHASE 0 — LEARN BY DOING (WEEKS 1-3)

The goal is to build hands-on intuition for the problem before writing any framework code.

Step 1 — Complete the reading map. Categories A and B in Week 1, C and D in Week 2. Everyone reads ContraGen in full — reviewers will compare DANDI to it.

Step 2 — Set up the three real services. Create a free Slack workspace (call the fake company “SynCorp”), a free HubSpot CRM account (or Zoho if HubSpot’s API limits are too restrictive), and a Google Drive folder with shared docs. These become the backbone of SynCorp-2026.

Step 3 — Create contradiction scenarios manually. Each team member populates data across all three systems to create one contradiction scenario. Example: Team member 1 creates a scenario where a customer was promised a feature in Slack, but the CRM shows no feature request, and the contract in Drive explicitly excludes custom features.

Step 4 — Write raw API scripts. Each team member writes a Python script that calls all three APIs, retrieves relevant records for their scenario, and prints them. No LLM involved yet — just understand the data shapes, API quirks, rate limits, and auth flows.

Step 5 — Baseline prompt experiment. As a team, write a prompt that, given the retrieved data as context, can identify the contradiction. Document what works, what fails, and why. This becomes the baseline.

Deliverables: Three working API wrappers (Slack, CRM, Drive) in Python. Three manually created contradiction scenarios with ground truth. Baseline prompt + results document. A shared document listing every API pain point, rate limit, and data shape surprise.

Key Decision: Which CRM (HubSpot vs. Zoho vs. alternative)? Decide based on free-tier API limits discovered during this phase.

Risk: API rate limits on free tiers may be more restrictive than expected. Mitigation: test limits early, have a backup CRM option ready.

02 PHASE 1 – CONTEXT SHELL SPECIFICATION AND IMPLEMENTATION (WEEKS 4-7)

The goal is to define and implement the Context Shell — the navigation interface that wraps real systems into a unified traversable substrate.

Step 1 — Formalize the six primitives based on what you learned in Phase 0. `list_sources` returns available systems with metadata (type, capabilities, last sync). `query(source, params)` issues a structured query against a specific source (e.g., search Slack for messages from user X mentioning “migration”). `open(source, id)` retrieves a specific record by ID. `search(query)` does cross-source semantic search — this is the expensive one, decide how to implement. `trace_link(source, id, direction)` follows a reference from one source into another (e.g., a Slack message mentions a Jira ticket ID → retrieve that ticket). `write_inference(evidence[], conclusion, confidence)` writes an inferred-state node.

Step 2 — Implement source wrappers. Implement a wrapper for each of the three real services that maps API responses into a common record format. The key design challenge: records from Slack (unstructured messages), CRM (structured fields), and Drive (documents) have very different shapes. Define a minimal common schema that preserves source-specific detail while enabling cross-source comparison.

Step 3 — Implement the navigation budget model. The agent gets N steps (start with 20, tune later). Each primitive call costs 1 step. The agent must decide how to allocate its budget strategically.

Step 4 — Implement `write_inference` storage. Start simple — inferred nodes stored as JSON files or in a lightweight SQLite database. Each node contains: evidence list (source + record ID + relevant excerpt), conclusion text, confidence score, timestamp, and the navigation path that led to it.

Step 5 — Implement `trace_link`. This is the hardest primitive — it requires recognizing cross-system references. Start with simple heuristics: regex for ticket IDs, email matching for person resolution, date-range correlation. Don’t over-engineer it.

Step 6 — Write integration tests. Can the shell navigate a known scenario end-to-end using only the six primitives?

Deliverables: Context Shell spec document (the primitives, their signatures, the navigation budget model). Working implementation wrapping three real services. Integration tests passing against the Phase 0 scenarios. `write_inference` storage with evidence chain format.

Compute Cost: Minimal — API calls only, no LLM usage yet.

Key Decision: Common record schema design. Keep it minimal. Don't try to build an ontology — that's the Builder team's problem. You need just enough structure to enable cross-source comparison.

03 PHASE 2 — DANDI AGENT IMPLEMENTATION (WEEKS 8-11)

The goal is to build the agent that uses the Context Shell primitives to navigate enterprise state, detect contradictions, and produce reconciled inferences.

Step 1 — Choose the LLM backbone. Recommendation: start with Claude Sonnet (good reasoning, affordable) for development. Run final benchmarks on both Sonnet and a frontier model to show scaling behavior. Budget approximately \$500 of API credits for development + benchmarks.

Step 2 — Implement the agent loop. Agent receives a query (e.g., “What migration terms were agreed with customer X?”). Agent calls `list_sources` to understand what's available. Agent plans a navigation strategy within its budget. Agent executes primitives, accumulating evidence. At each step, agent evaluates: do I have enough evidence? Do I see a contradiction? If contradiction detected across 3+ sources: agent calls `write_inference` with evidence chain and reconciliation. Agent returns final answer with evidence trail.

Step 3 — Implement the discrepancy detection prompt. The agent needs to be explicitly prompted to look for conflicts, not just answer the question. Design the system prompt to include instructions like: “Before answering, check whether your evidence sources agree. If they disagree, identify the specific claims that conflict and which sources support each claim.”

Step 4 — Implement the evidence chain format. Each `write_inference` call must produce: what sources were consulted, what each source claimed, where they conflict, what the reconciled conclusion is, and the confidence level.

Step 5 — Test against the Phase 0 scenarios. The agent should find all three manually created contradictions. If it doesn't, iterate on the prompt and the navigation strategy.

Step 6 — Run ablations. Agent with all 6 primitives vs. agent with only `search` (simulates RAG). Agent with `write_inference` vs. without (does write-back improve subsequent queries?). Navigation budget of 10 vs. 20 vs. 40 steps. Effect of `trace_link` — does cross-system link following matter?

Deliverables: Working DANDI agent that navigates real systems and detects contradictions. Ablation results document. Evidence chain examples from each test scenario.

Compute Cost: Approximately \$300-500 in LLM API calls for development and testing.

Risk: The agent may burn its entire navigation budget without finding the contradiction. Mitigation: implement a “guided exploration” mode where the agent is given the source types involved (but not the specific records) as a hint. Compare guided vs. unguided performance.

04 PHASE 3 — SYNCORP-2026 BENCHMARK (WEEKS 10-13)

The goal is to build a reproducible benchmark environment with enough scale and variety to be a credible evaluation. Overlaps with Phase 2 — start SynCorp design while agent development is in progress.

Step 1 — Design the SynCorp company. Create a fictional mid-size SaaS company with approximately 10 customers, 5 products/plans, and 20 employees. This determines the data volume.

Step 2 — Design contradiction categories. Five types: promise vs. contract (Slack says X, contract says Y), billing vs. plan (charge doesn’t match recorded plan), timeline conflicts (CRM says closed in March, contract dated April), authority conflicts (rep promised discount they couldn’t authorize), and status conflicts (ticket marked resolved, customer still complaining in Slack).

Step 3 — Create scenario instances. For each category, create at least 3 instances — 15 minimum test cases. Each instance involves data planted in all three systems. Document the ground truth for each.

Step 4 — Seed the real services. Write scripts to seed Slack messages, CRM records, and Drive documents programmatically via APIs. Populate the real services with the SynCorp data.

Step 5 — Design the evaluation metrics. Discovery rate: did the agent find the contradiction? (binary per scenario). Evidence completeness: did the agent cite all relevant sources? (recall). Reconciliation accuracy: is the inferred conclusion correct? (judged against ground truth). Token efficiency: total tokens consumed vs. long-context baseline. Navigation efficiency: steps used vs. budget allocated. Path quality: did the agent follow a reasonable navigation path? (human evaluation).

Step 6 — Implement and run baselines. Long-context baseline: dump all records from all three systems into a single prompt, ask the same question. RAG baseline: embed all records, retrieve top-k, ask the question. DANDI without SSR: Context Shell navigation but no `write_inference`. DANDI full: Context Shell + SSR. Run all baselines and DANDI against all 15+ scenarios. Collect metrics.

Deliverables: SynCorp-2026 data design document. Seeding scripts for all three systems. Ground truth annotations for all scenarios. Evaluation harness code. Results table comparing all baselines.

Compute Cost: Approximately \$200-400 for baseline runs (long-context baseline is the expensive one).

Key Decision: How many scenarios is enough? 15 is the minimum for a credible paper. 30 would be strong. Depends on how fast data seeding goes.

05 PHASE 4 — PAPER WRITING AND RELEASE (WEEKS 13-18)

The goal is to write the paper, prepare the open-source release, and submit.

Write the paper. Structure: Introduction (the context graph wave, the missing reliability layer, our contribution) → Related Work (context graphs, GraphRAG, ContraGen, agent memory) → Method (Context Shell primitives, SSR protocol, DANDI agent architecture) → SynCorp-2026 (benchmark design, contradiction taxonomy, setup instructions) → Experiments (baselines vs. DANDI, ablations, qualitative examples) → Discussion (limitations, future work including the Builder + Walker pipeline) → Conclusion.

Prepare the SynCorp-2026 release package. Seeding scripts, evaluation harness, setup instructions, ground truth (held-out test set kept separate from training/dev examples).

Prepare the DANDI agent release. Context Shell spec, wrapper implementations, agent code, configuration for different LLM backends.

Internal review. Each team member reviews another's section. Run the evaluation one more time to confirm numbers.

Target venue. Decide based on timeline. Options include EMNLP, ACL, NAACL (NLP venues), AAAI, NeurIPS workshops, or industry-focused venues like CIKM or KDD. An arXiv preprint first to claim the space, then submit to a venue.

Deliverables: Paper draft (LaTeX). arXiv preprint. GitHub repo with SynCorp-2026 + DANDI agent. HuggingFace dataset card for SynCorp-2026.

06 TIMELINE SUMMARY

PHASE	WEEKS	FOCUS
Phase 0	1-3	Learn by doing — APIs, data shapes, baseline prompt
Phase 1	4-7	Context Shell specification and implementation
Phase 2	8-11	DANDI agent implementation and ablations
Phase 3	10-13	SynCorp-2026 benchmark (overlaps Phase 2)
Phase 4	13-18	Paper, release, submission

Total: approximately 5 months with a team of 3. Phases 2 and 3 overlap intentionally — SynCorp design informs agent testing, and agent testing surfaces benchmark design issues.

07 COST ESTIMATE

ITEM	ESTIMATED COST
LLM API calls (development)	\$300-500
LLM API calls (benchmarks)	\$200-400
Free-tier services (Slack, CRM, Drive)	\$0
Hosting/compute	\$0 (local development)
Total	\$500-900

08 RISK REGISTER

RISK	IMPACT	LIKELIHOOD	MITIGATION
Free-tier API rate limits too restrictive	High	Medium	Test limits in Phase 0, have backup services ready
Agent can't reliably detect contradictions	High	Medium	Iterate on prompts, add guided mode, consider multi-pass
SynCorp scenarios feel too synthetic	Medium	Medium	Use realistic enterprise language, get practitioner feedback
ContraGen team publishes multi-source version	Medium	Low	Differentiator is real APIs + write-back, maintain this gap
Scope creep into ontology/graph construction	High	Medium	Strict boundary: shell navigates systems as-is

SECTION 05

IP Suitability Note

01 NOVELTY ASSESSMENT

The primary patentable contribution is Semantic State Refactoring (SSR) — a method in which an AI agent, during navigation of heterogeneous enterprise data sources, detects discrepancies across three or more systems and writes reconciled inferred-state nodes back into a persistent substrate with explicit evidence chains.

Claim 1 — The `write_inference` primitive and its execution protocol

An agent operating within a constrained navigation interface over heterogeneous enterprise systems (messaging APIs, CRM systems, document stores) detects a contradiction spanning $N \geq 3$ sources, generates a reconciled conclusion with a structured evidence chain (source identifiers, specific claims per source, conflict description, resolution), and writes this as a persistent node that is queryable by downstream agents. The key novelty: the agent transforms read-only enterprise state into a self-refactoring substrate where inference artifacts accumulate and reduce future query cost.

Claim 2 — Bounded navigation budget model applied to cross-system enterprise traversal

An agent is allocated a fixed number of navigation steps across heterogeneous real systems (not a pre-built knowledge graph) and must strategically allocate its budget to maximize contradiction discovery. The combination of bounded traversal + contradiction detection + write-back over real system APIs (not graph databases) is novel.

Claim 3 — The discrepancy test formalization

A specific evaluation method requiring resolution of a hidden truth that can only be determined by evidence from three or more heterogeneous enterprise platforms, where no single platform contains the complete answer and at least two platforms contain contradictory claims.

Prior Art Landscape

GraphRAG (Microsoft) traverses pre-built knowledge graphs for multi-hop QA. Read-only — no write-back or contradiction-aware refactoring. Operates on a graph DB, not real heterogeneous APIs.

Graphiti/Zep does write-back to knowledge graphs but focuses on temporal memory accumulation (facts that change over time), not contradiction-aware reconciliation across disparate sources.

ContraGen generates contradictory documents for evaluation but works at the document level with NLI-based detection, not agent navigation across real heterogeneous systems.

LangGraph provides stateful agent orchestration but does not define navigation primitives over enterprise systems or write-back mechanisms for inferred state.

No identified prior art combines all three elements: (a) navigation over real heterogeneous enterprise APIs through a standardized primitive set, (b) active contradiction detection across 3+ sources, and (c) write-back of reconciled inference nodes with evidence chains.

02 OBVIOUSNESS POSTURE

The combination is non-obvious for three reasons.

First, the prevailing industry approach treats enterprise context as a retrieval problem (RAG, GraphRAG) or an ingestion problem (long-context windows). Nobody has proposed that agents should actively refactor enterprise state by writing reconciled inference nodes back into the substrate. The field's assumption is that context is read-only — DANDI challenges this.

Second, contradiction detection in prior art (ContraGen, Amazon's RAG contradiction work) operates on documents, not across live heterogeneous APIs. Extending detection from document-level NLI to real-time cross-system navigation with write-back is a substantial architectural shift, not an obvious extension.

Third, the bounded navigation budget forces strategic traversal rather than exhaustive search. This constraint — combined with contradiction detection and write-back — creates a novel optimization problem: how to maximally discover and reconcile conflicts within a fixed number of API calls across systems with different data shapes and access patterns. No prior work addresses this.

Indian Patent Act Considerations

Section 3(k) — “computer programme per se” exclusion. DANDI’s claims should be framed around the technical effects, not the software:

- A method for detecting and reconciling contradictory state across heterogeneous enterprise data systems (technical effect: improved accuracy of cross-system information retrieval, measurable via contradiction detection rate)
- A method for producing persistent inference artifacts from cross-system navigation (technical effect: reduced token consumption on future queries, measurable reduction in redundant computation)
- A system involving interaction with real external systems (APIs), not purely abstract computation (technical effect: tangible state modification across real enterprise systems)

Frame every claim around the technical process and its measurable effect. The method involves interaction with real external systems (APIs), which strengthens the “not software per se” argument.

Filing Strategy

Step 1 — Provisional application (Indian Patent Office). File within 30 days of brief approval. Cost: approximately Rs 1,600 for a startup entity. A provisional gives 12 months to file the complete specification. This locks the priority date. OG Labs’ Section 8 status may qualify for reduced fees.

Step 2 — Formal prior art search. Conduct a focused search on patent databases (Google Patents, IPO India) for combinations of “enterprise system navigation,” “contradiction detection,” and “write-back” or “inference storage.” Can be done by the team, not a lawyer.

Step 3 — Complete specification. File within 12 months of provisional. Include experimental results from Phases 2-3 as evidence of technical effect — the discovery rate tables, token efficiency comparisons, and ablation results are the strongest evidence.

Step 4 — PCT route (optional). If results are strong and there is industry interest, file a PCT application within 12 months. Cost: approximately \$3,000–5,000. May be outside budget; pursue only if a clear commercialization path materializes (licensing to enterprise AI platforms, integration into agent orchestration frameworks).

Commercial Relevance

Any company building AI agents for enterprise workflows is a potential user or licensee. The Context Shell specification could become a standard interface for enterprise agent navigation — similar to how LSP (Language Server Protocol) standardized how editors interact with language tools. Specific verticals: customer support platforms, sales intelligence tools, compliance and audit systems, and the growing ecosystem of enterprise AI agent frameworks.

04 RECOMMENDATION

File a provisional patent application in India before any public disclosure. The provisional is cheap (Rs 1,600), fast (2–3 days with the Manipal University IP team), and preserves all options. Do not publish the arXiv preprint, submit to a conference, or release the GitHub repo until the provisional is filed.

Recommended claims structure:

- **Method claim:** A computer-implemented method for detecting and reconciling contradictory state across heterogeneous enterprise data systems, comprising the steps of navigation via a standardized primitive set, contradiction detection across 3+ sources, evidence chain assembly, and write_inference execution to produce persistent reconciled state nodes.
- **System claim:** A system comprising a Context Shell interface layer exposing a plurality of enterprise data sources through a unified set of navigation primitives, an agent module that traverses said sources under a bounded navigation budget, and an inference storage module that persists reconciled state nodes.
- **Benchmark claim (weaker but worth including):** A method for evaluating cross-source contradiction resolution capability of AI agents using a test harness comprising real heterogeneous enterprise service instances populated with injected multi-source conflicts.

SECTION 06

Open Source Release Blueprint

01 RELEASE SCOPE

Everything is open. Nothing is closed. The entire contribution ships on the day of paper release.

SynCorp-2026 benchmark — seeding scripts, evaluation harness, ground truth annotations, setup guide. The benchmark's credibility depends on full reproducibility.

Context Shell specification — primitive definitions, interface contracts, navigation budget model. Published in the repo's `/spec` directory alongside the reference implementation.

Context Shell reference wrappers — Slack, CRM (HubSpot/Zoho), Google Drive. These are the source wrappers that map real APIs into the shell's common record format.

DANDI agent reference implementation — the agent loop, discrepancy detection prompt, evidence chain format, and `write_inference` execution.

Discrepancy test harness — evaluation scripts that score agents on discovery rate, evidence completeness, reconciliation accuracy, token efficiency, and navigation efficiency.

Evaluation results and analysis scripts — all numbers from the paper, reproducible from the released code.

Paper — arXiv preprint linked from the repo.

02 CONSTRAINTS

All code must run on a clean machine with only documented dependencies. No hardcoded paths, API keys, or personal data in the code or git history (run a secrets scanner before release). SynCorp setup should take no more than 30 minutes for someone following the guide. Evaluation results must be reproducible from the released code — spot-check at least 3 key numbers before release.

03 READINESS EXPECTATIONS

Release Checklist

Before release, all of the following must be true:

- All code runs on a clean machine with only documented dependencies

- README includes: what DANDI is, why it matters (2 paragraphs max), quickstart (set up SynCorp in 30 minutes), full setup guide, citation info

- SynCorp setup guide tested by someone who wasn't involved in building it

- HuggingFace dataset card filled out (description, intended use, limitations, citation)

- LICENSE file in repo root (Apache 2.0)

- All API keys/secrets removed from code and history

- Evaluation results reproducible from released code

- arXiv preprint uploaded and linked in README

- CONTRIBUTING.md with guidelines for adding new source wrappers, new contradiction scenarios, and running the harness against custom agents

- CITATION.bib file

At release: GitHub repo set to public, HuggingFace dataset published, blog post on ongroundlabs.org, Twitter/X thread, posts to r/MachineLearning, r/LocalLLaMA, Hacker News, and relevant Discord servers.

After release: monitor GitHub issues for first 2 weeks (one team member as triage lead), respond to arXiv comments, track citations and forks.

04 RELEASE PLATFORMS

ARTIFACT	PLATFORM	REASON
All code (agent, shell, harness)	GitHub (og-labs/dandi)	Standard for code, issues and PRs for community
SynCorp-2026 dataset + annotations	HuggingFace Datasets	Standard for ML benchmarks, easy to load
Paper	arXiv	Standard for preprints
Context Shell spec	GitHub (same repo, /spec directory)	Keeps spec and implementation together
Blog post / announcement	ongroundlabs.org + Twitter/X	Reach the practitioner audience

05 LICENSING

Apache 2.0 for all code (Context Shell, agent, evaluation harness, seeding scripts). Apache 2.0 allows commercial use (important for adoption), includes a patent grant (relevant given the IP filing), and is the standard for enterprise-facing open-source AI tooling. MIT is simpler but lacks the patent grant. GPL would limit enterprise adoption.

CC BY 4.0 for the SynCorp-2026 benchmark data, ground truth annotations, and the Context Shell specification document. Standard license for research datasets and specifications — requires attribution but allows any use.

06 RECOMMENDATION

Release fully open-source from day one. Apache 2.0 for code, CC BY 4.0 for data and specs. The patent protects the method; open release maximizes adoption, citation, and community contribution.

Community and Maintenance Plan

Maintenance commitment: 6 months active maintenance post-release. Bug fixes, documentation improvements, and community PRs reviewed within 1 week.

Contribution guide (CONTRIBUTING.md): how to add a new source wrapper to the Context Shell (e.g., wrapping Jira, Notion, or another CRM), how to add new contradiction scenarios to SynCorp-2026, and how to run the evaluation harness against your own agent.

Growth targets (realistic for an academic lab): 50 GitHub stars in first month, 3 external reproductions of SynCorp in first 3 months, 1 external paper citing DANDI or SynCorp in first 6 months.

Long-term: if the community grows, consider organizing a shared task or workshop around SynCorp-2026 at an NLP or IR venue. This is how benchmarks become adopted — through community participation, not just publication.

Naming note: “DANDI” has some namespace conflicts in software (Dandi AI for HR analytics, DANDI Archive for neurophysiology). For the GitHub repo and public-facing tooling, use “og-labs/dandi” to distinguish. The benchmark should always be referred to as “SynCorp-2026” which is clean and unambiguous.

SECTION 07

Appendix

01 DECISION LOG

▪ D1 — CRM CHOICE

HubSpot free tier vs. Zoho CRM vs. alternative. Decide in Phase 0 based on API rate limits, data model richness, and ease of programmatic seeding. HubSpot has better brand recognition (helps with paper credibility) but may have tighter free-tier limits. Test both.

▪ D2 — COMMON RECORD SCHEMA

The Context Shell needs a minimal common schema that unifies Slack messages (unstructured), CRM records (structured fields), and Drive documents (semi-structured). Keep it minimal — don't build an ontology. Enough structure for cross-source comparison: record ID, source type, timestamp, content (text), entities mentioned (people, deals, products), and metadata (author, channel, status). Decision in Phase 1 Step 2.

▪ D3 — `TRACE_LINK` IMPLEMENTATION

Start with simple heuristics: regex for ticket IDs, email matching for person resolution, date-range correlation. Don't over-engineer — the Phase 0 exercise will reveal what cross-system references actually look like in practice. Upgrade to entity resolution if heuristics prove insufficient.

▪ D4 — `WRITE_INFERENCE` STORAGE

Start with JSON files or lightweight SQLite. Each node: evidence list, conclusion, confidence, timestamp, navigation path. Upgrade to a proper store only if query performance becomes a bottleneck during benchmark runs. Decision in Phase 1 Step 4.

▪ D5 — NAVIGATION BUDGET DEFAULT

Start with 20 steps. Ablate at 10 and 40 in Phase 2 Step 6. The budget should be tight enough to force strategic traversal but loose enough that the agent can find 3-source contradictions. 20 is a hypothesis — the ablation results will confirm or revise it.

- **D6 — LLM BACKBONE**

Claude Sonnet for development (affordable, good reasoning). Run final benchmarks on both Sonnet and a frontier model (Claude Opus or GPT-4) to show scaling behavior. Budget \$500 for development + benchmarks. Decision confirmed in Phase 2 Step 1.

02 RUBRIC AND EVALUATION CLARIFICATIONS

DANDI uses six evaluation metrics, not a single composite score. The metrics capture different aspects of cross-system contradiction resolution.

Discovery Rate (Primary) Binary per scenario: did the agent identify the contradiction? A “discovery” requires the agent to explicitly state that sources disagree and name the specific conflicting claims. Simply answering the question correctly without noting the conflict does not count as discovery.

Evidence Completeness Recall metric: of the sources involved in the contradiction (minimum 3), how many did the agent cite in its evidence chain? Full marks require citing all relevant sources with the specific claims from each.

Reconciliation Accuracy Judged against ground truth: is the agent’s reconciled conclusion correct? This is a human judgment call — the evaluator checks whether the reconciliation matches the injected ground truth and whether the reasoning is sound.

Token Efficiency Total tokens consumed (input + output) to resolve the query, compared against the long-context baseline. The hypothesis is that DANDI uses fewer tokens because it navigates strategically instead of ingesting everything. Report as a ratio: DANDI tokens / baseline tokens.

Navigation Efficiency Steps used vs. budget allocated. An agent that finds the contradiction in 8 steps out of 20 is more efficient than one that uses all 20. Report both the raw step count and the ratio.

Path Quality (Human Evaluation) Did the agent follow a reasonable navigation path? A human evaluator reviews the sequence of primitive calls and judges whether the traversal was strategic (following leads, cross-referencing sources) or random (blindly querying sources with no apparent strategy). Rated on a 3-point scale: strategic, mixed, random.

03 DATA AND SCHEMA CLARIFICATIONS

SynCorp-2026 Company Profile Fictional mid-size SaaS company. Approximately 10 customers, 5 products/plans, 20 employees. Data volume is modest — the benchmark tests navigation strategy, not scale.

Contradiction Categories

CATEGORY	EXAMPLE	SOURCES INVOLVED
Promise vs. contract	“Free migration” in Slack, no mention in contract	Slack + Drive + CRM
Billing vs. plan	Enterprise charge after downgrade to Pro	CRM + billing + support ticket
Timeline conflicts	CRM says March close, contract dated April	CRM + Drive
Authority conflicts	Rep promised discount they couldn’t authorize	Slack + CRM + Drive
Status conflicts	Ticket marked resolved, customer still complaining	CRM (ticket) + Slack

Minimum 3 instances per category = 15 test cases. Target 30 for a strong paper.

Ground Truth Format Each scenario has: the injected contradiction (what specifically conflicts), the ground truth resolution (what actually happened), the evidence chain (which records in which systems contain the relevant claims), and the minimum navigation path (the shortest sequence of primitive calls that would discover the contradiction).

04 OPERATOR RUNBOOK NOTES

Setting up SynCorp-2026. The benchmark requires three real service accounts: a free Slack workspace, a free CRM account (HubSpot or Zoho), and a Google Drive folder. Seeding scripts populate these with the SynCorp data. Setup should take approximately 30 minutes. Document every step — the setup guide is part of the release.

Running the evaluation. The test harness takes an agent implementation (anything that can call Context Shell primitives) and runs it against all SynCorp scenarios. For each scenario: invoke the agent with the query, capture the full primitive call trace, collect the agent’s answer and evidence chain, score against ground truth. Run each scenario twice to measure non-determinism.

API key management. The three services require API keys/tokens. Store them in environment variables, never in code. The harness reads from env vars. Document the required env vars in the README.

Rate limit handling. Free-tier rate limits vary by service. Implement response caching in the Context Shell wrappers — if a record was fetched in the last N minutes, return the cached version instead of making another API call. This is also realistic: production deployments would cache aggressively.

05 COST CONTROL APPENDIX (MANDATORY)

Total budget: under \$1,000 for the full project.

LLM API Calls Development (Phase 2): approximately \$300-500. Benchmarks (Phase 3): approximately \$200-400. The long-context baseline is the most expensive single experiment — dumping all SynCorp data into a single prompt for each of 15-30 scenarios at frontier model prices.

Free-Tier Services Slack, HubSpot/Zoho, and Google Drive: \$0. All three offer free tiers with sufficient API access for the benchmark. If HubSpot’s limits are too tight, Zoho is the fallback.

Hosting/Compute Local development: \$0. The evaluation harness runs on any machine with Python and Docker. No cloud infrastructure needed.

Fallback If LLM API costs exceed \$500 during development, switch to a cheaper model for iteration and reserve the frontier model for final benchmark runs only. If free-tier limits are hit during benchmark runs, add caching and batching to reduce API calls.

06 OPEN QUESTIONS

- **SEARCH IMPLEMENTATION**

Cross-source semantic search is the most expensive primitive. Options: embed all records and use vector similarity (requires an embedding step), use each service's native search and merge results (faster but less semantic), or skip cross-source search entirely and rely on `query` + `trace_link` for navigation. The choice affects both cost and the paper's claims about the primitive set.

- **WRITE-BACK SAFETY**

Hallucinated inference nodes can poison future queries. What confidence threshold is required for `write_inference` ? Should nodes be retractable if later evidence contradicts them? Should there be a garbage collection mechanism for stale inferences? These are important for the paper's discussion section but may be out of scope for v0.1.

- **SCENARIO COUNT**

15 scenarios is the minimum for a credible paper. 30 is strong. 50 would be exceptional but requires significant data seeding effort. Where's the sweet spot given the team size and timeline?

- **BENCHMARK REPRODUCIBILITY ACROSS TIME**

Free-tier API behavior may change over time. Slack, HubSpot, and Google may alter rate limits, deprecate endpoints, or change free-tier features. Should the benchmark include a "snapshot mode" that works from cached API responses for reproducibility? This trades realism for stability.

- **BUILDER + WALKER PIPELINE**

The source material mentions "Agent Builder" as a separate team responsible for ontology/graph construction. DANDI (the Walker) navigates systems as-is. A future integration could have the Builder construct a context graph and the Walker navigate it. Is this future work worth mentioning in the paper's discussion section, or does it muddy the contribution?